25 全點對最短路徑

Part VI Graph Algorithms

本章定位

全點對最短路徑有兩種不同DP狀態：邊數與中介點；本章推導min-plus、Floyd與Johnson，連結可重建路徑、零權圈、負環影響範圍及重加權的正負號與成本。

前置知識

矩陣最小值加法與前驅表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 把 min-plus 矩陣乘法接到最短路徑。
- 用 Floyd-Warshall 的中介點 DP 推導轉移。
- 用 Johnson 重加權處理負邊與稀疏圖。
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP（全點對最短路徑）的輸入是 n 個頂點的有向加權圖，邊權可為負。輸出每一對 (i,j) 的距離；沒有路徑為 $+\infty$。若存在 i 可達、且能走到 j 的負 cycle，任意重走該圈使成本無下界，這一對的距離定為 $-\infty$，並沒有可交付的有限最短路徑。

本節先假設無負 cycle。任何重複頂點的 walk 都包含非負 cycle，刪圈不增加成本；可反覆刪到 simple path。因此 $i \neq j$ 的最短路徑可取至多 $n-1$ 邊，$i=j$ 則可取零長路徑、成本0。

權矩陣 W[i,j] 是直接邊最小權，無邊為∞；W[i,i]=0 表示可不走邊。若要偵測負 self-loop，初始化必取 min(0,w(i,i))，不能把負 self-loop 覆蓋成0。下面的無負環數例對角線均0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

以最多邊數為狀態

令 $L^{(m)}[i,j]$ 為 i 到 j **最多 m 條邊的 walk** 最小權重；不是恰好 m 條邊。$L^{(0)}=I$，其中 I[i,i]=0、非對角為∞。按最後一條邊拆解，有

$L^{(m+1)}[i,j]=\min_k(L^{(m)}[i,k]+W[k,j])$。

任何非空候選可拆成最多 m 邊前綴加最後一邊；反過來每個有限項都可實際串接。較短的舊路徑也保留，因 k=j 時 W[j,j]=0。因此兩方向涵蓋同一組可行 walk，這是轉移等式的理由。即使有負 cycle，固定 m 的定義仍有限且有意義；只是不能在 m=n−1 宣稱已得無限制的最短距離。

定義 min-plus product：$(A\otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$。若 A、B 分別對應最多 a、b 邊，串接最多 a+b 邊；反向沿任一最多 a+b 邊 walk，在第 min(a,實際長度) 邊切開即可，故 $L^{(a)}\otimes L^{(b)}=L^{(a+b)}$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Min-plus 的代數與兩種算法

結合律來自同一個有限最小值：$((A\otimes B)\otimes C)[i,j]=\min_{p,\varphi}(A[i,p]+B[p,q]+C[q,j])=(A\otimes(B\otimes C))[i,j]$。I 是單位矩陣；∞是 min 的中性元，∞+有限值=∞。本階段不把−∞放入乘法，以免遇到∞+(−∞)沒有定義。

```
Extend(A,B):
  C=全∞的新 n×n 矩陣
  for i,j,k: C[i,j]=min(C[i,j],A[i,k]+B[k,j])
  return C
Slow(W): L=I; 重複n-1次 L=Extend(L,W); return L
Square(W): L=W; m=1
  while m<n-1: L=Extend(L,L); m=2m
  return L
```

Slow 做 $\Theta(n)$ 次 $\Theta(n^3)$ product，$n\geq 2$ 時 $\Theta(n^4)$；Square 做 $\text{ceil}(\log_2(n-1))$ 次 squaring，$O(n^3 \log n)$。保留兩個輸入與一個新輸出即可 $O(n^2)$ 工作空間；不能一邊覆寫輸入而仍宣稱每次 product 精確對應同一舊邊數。n=0回空矩陣，n=1直接處理對角線。

普通乘法把 min 換成加法、把加法換成乘法，但 Strassen 的減法公式不能直接套入 min-plus；上述類比並未證明一般環上的快速乘法界也成立。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

四點六邊的共同數例

以下全部矩陣、Floyd stages 與 Johnson 都使用：1→2:4、1→3:7、2→3:−2、2→4:6、3→4:3、4→2:1。頂點1無入邊，其餘三點 cycle 2→3→4→2 成本2，沒有負 cycle。

W 的 rows 為 [0,4,7,∞]、[∞,0,−2,6]、[∞,∞,0,3]、[∞,1,∞,0]。

$L^{(2)}$ 的 rows 為 [0,4,2,10]、[∞,0,−2,1]、[∞,4,0,3]、[∞,1,−1,0]。

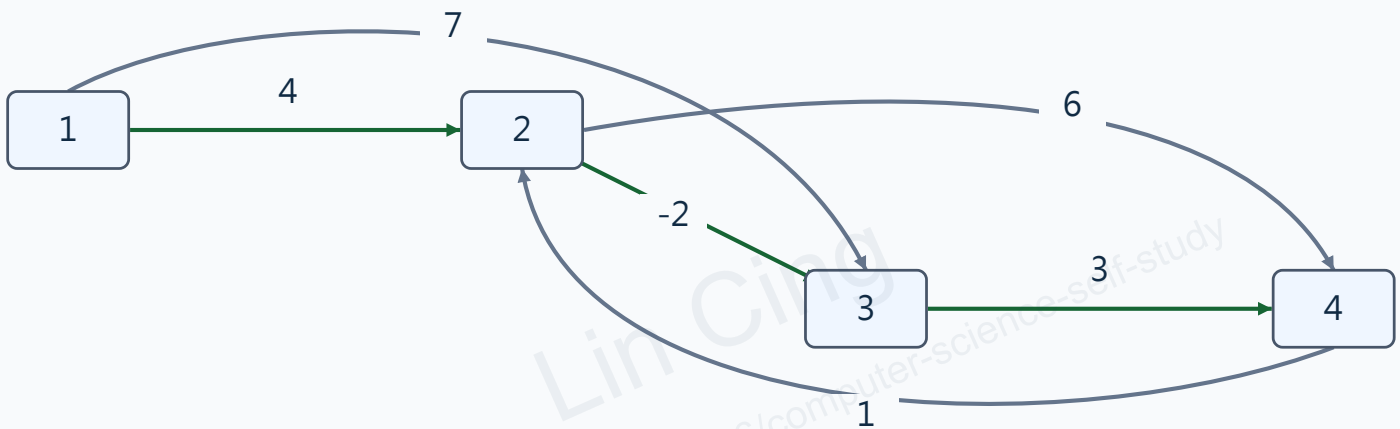
例如 $L^{(2)}[1,4]=\min(0+\infty,4+6,7+3,\infty+0)=10$，兩條2邊路徑並列；$L^{(2)}[2,4]=-2+3=1$。第三次 Extend 才允許1→2→3→4，$L^{(3)}[1,4]=5$。其餘格不變；n=4，所以 $L^{(3)}$即答案，squaring 的 $L^{(4)}$也相同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

四點六邊的共同數例 （續）

六條有向邊與邊數受限的路徑

綠線 = 最佳3邊路徑；灰線 = 其餘原圖邊。邊旁數字是原權重。



1→2→4=10（2邊）；1→3→4=10（2邊）；1→2→3→4=5（3邊）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.2 The Floyd-Warshall algorithm

Floyd–Warshall 的 k 是**允許的中介頂點編號上界**，不是允許的邊數。$D^{(k)}[i,j]$ 是所有中介點在 $\{1,\cdots,k\}$ 的 $i\to j$ walk 最小成本；端點 i,j 不受這個集合限制。無負 cycle 時可取 simple path，因此 k 最多出現一次。

最優路徑若不經 k，屬於舊狀態 $D^{(k-1)}$；若經 k，在 k 切開，兩段內部不含 k，因此候選為 $D^{(k-1)}[i,k]+D^{(k-1)}[k,j]$。最優子結構可用替換論證：若任一子段有更便宜的合法替代，串接後成本更小，刪除非負 cycle 也不變差，與最優矛盾。反向串接兩個合法舊子路徑只增加可用中介點 k，故不會取得比狀態定義還小的不合法成本。

所以 $D^{(0)}=W$，且 $D^{(k)}[i,j]=\min(D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j])$。到 k=n 全部中介點皆允許，得到 APSP。逐 k 建新矩陣需 $\Theta(n^3)$ 時間；保留全部 stages 需 $\Theta(n^3)$ 空間，僅留舊／新兩份則 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

原地更新為何成立；Loop Order 為何不能任換

無負 cycle 時，每個 stage 的 D[k,k]=0。固定 k 更新 row k 時，候選 D[k,k]+D[k,j]=D[k,j]，不會改值；column k 同理。因此本輪所有 D[i,k]、D[k,j] 都與上一輪相同，各 D[i,j] 一次更新恰實作原轉移，可以只用一個矩陣。

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i,k]!=∞ and D[k,j]!=∞:
        D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

$\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。有限整數實作須避免INF sentinel加負數變成「可達」，以及加法溢位；上面的可達判斷先做，精確算術的數學定理才適用。

反例：單位權鏈1→2→4→3，若依 i,j,k 順序只跑一遍，處理 i=1,j=3 時 D[1,4] 與 D[2,3] 都尚為∞，所以 D[1,3] 留∞；稍後 j=4 才算出 D[1,4]=2，卻不會回頭更新 j=3。正確 k,i,j 得 D[1,3]=3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個 k 的完整矩陣

rows/columns 都依1,2,3,4。$D^{(0)}=W$；k=1 因1無入邊，不新增路徑，$D^{(1)}=W$。

stage	row1	row2	row3	row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

k=2 更新三格：(1,3)取4−2=2、(1,4)取4+6=10、(4,3)取1−2=−1。k=3 更新(1,4)取2+3=5、(2,4)取−2+3=1。k=4 更新(3,2)取3+1=4。其餘格的舊值不大於候選，所以保留。

對照 $L^{(2)}$：它已允許2→3→4與3→4→2，但 $D^{(2)}$尚不允許中介3或4，所以兩格分別仍是6與∞。兩者都叫「第二步」，卻不是同一狀態。

每個 k 的完整矩陣 （續）

Floyd 的六次更新：兩個輸入到一個結果

每列只記嚴格改善；右兩個輸入都來自上一個 stage。其餘格不變。

k	(i,j)	舊值	D[i,k]	D[k,j]	新值
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

綠格 = min後的新值；k=1沒有更新，k=2三格、k=3兩格、k=4一格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑輸出：前驅不是中介點

Π[i,j] 表示某條 i→j 最短路徑上，**緊接 j 之前**的點；不是任意中介 k。初始化直接邊 i→j 的 Π[i,j]=i；i=j或不可達為 NIL。採兩個 stage buffer 時，嚴格改善經 k 的值，就取 Π新[i,j]=Π舊[k,j]，因為最後一邊來自 k→j 的後半段；否則沿用舊值。k=j時候選不會嚴格改善，所以不會用空路徑的 NIL覆蓋合法前驅。

等成本時保留舊路徑，不可隨意把 < 改成 ≤。最小例：直接邊1→2權1，在k=2時候選1+0=1；若等值也更新Π[1,2]←Π[2,2]=NIL，就把原本可重建的路徑弄斷。對零權 cycle，更不能只從最後的「距離緊邊」任意選父而不證明無環。

本例Π[1,4]的嚴格改善：k=2時取Π[2,4]=2，暫得1→2→4；k=3時取Π[3,4]=3，且Π[1,3]=2、Π[1,2]=1，最後反向鏈4←3←2←1，成本4−2+3=5。下面給一個額外的 tie rule，使完整 predecessor tree 的無環性可直接由整數下降證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零權 Cycle 下可證無環的前驅建構

把每格改為 pair (cost,hops)，先比 cost，成本相同再選 hops 小者；兩段相加時兩欄分別相加。初始對角(0,0)、直接邊(w,1)、無路(∞,∞)，同樣做 Floyd轉移。無負 cycle 時，正權圈讓cost變差，零權圈讓hops變差，所以 lexicographic（字典序）最優者必有一條simple path，前述分解與原地證明均成立，仍為$\Theta(n^3)$次固定欄運算。

固定source s，對每個有限且$v \neq s$的v，從原圖入邊選一條$u \to v$，使

$cost[s,v]=cost[s,u]+w(u,v)$，$hops[s,v]=hops[s,u]+1$。

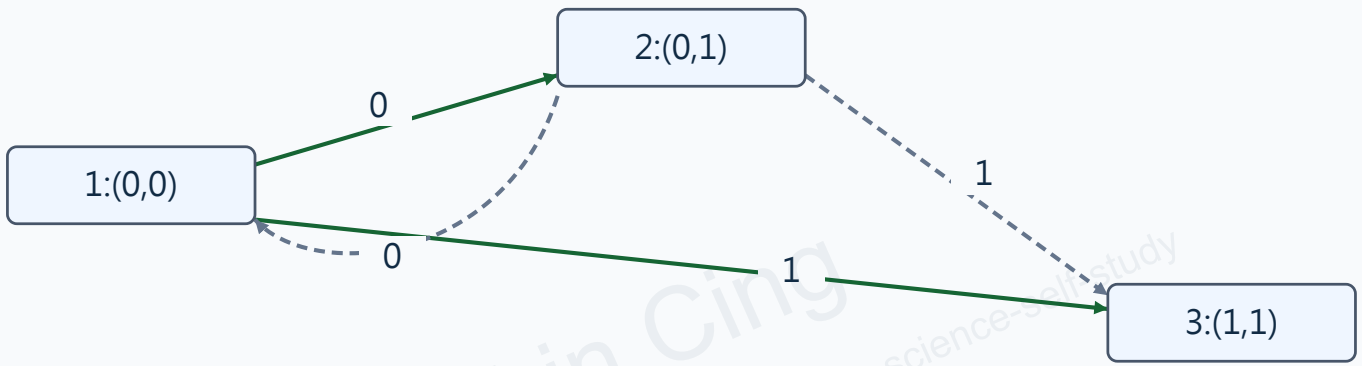
這樣的u必存在：取一條字典序最優$s \to v$路徑的最後一邊，若其前綴不是字典序最優，可替換而改善整條路徑，矛盾。設定$\Pi[s,v]=u$；沿parent每步hops嚴格減1，非負整數不可能無限下降，也不可能有cycle。除了s外有限點都有parent，因此必到s，得到一棵確實重建最短路徑的tree。

對每個s掃全部m邊即可選父，總$O(nm)$，簡單圖$m \leq n^2$時不超過Floyd的$\Theta(n^3)$；保存cost/hops/Π為$\Theta(n^2)$。例如$1 \leftrightarrow 2$皆權0、$1 \to 3$與$2 \to 3$皆權1，source1時hops=(0,1,1)，不能把1的parent設為2，亦不會在1、2之間循環。

零權 Cycle 下可證無環的前驅建構 （續）

零圈也能安全回溯：Hop 是嚴格下降量

格內 = 頂點:(cost,hops)；綠邊被選用，回溯時沿綠邊反向。



Π[1,2]=1、Π[1,3]=1；source1沒有parent。

2→3雖也給cost1，但要2 hops，輸給直接1→3的1 hop。

灰虛線 = 未選邊；每次parent回溯令hop少1，不會形成零權cycle。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

負 Cycle 偵測與受影響點對

若跑完Floyd有D[k,k]<0，該值由實際邊串接而成，代表負的closed walk；分解成simple cycles，至少一圈權重負。反向若有負simple cycle，取其最大編號的點k，其餘點都可作k→k中介，Floyd必能找出成本負的closed walk，故某對角線會負。這給偵測的兩方向。

有負環時，固定stage仍能產生一些有限walk值；但它們不能當作受影響pair的最短距離，上一頁依D[k,k]=0的原地等價論證也不適用。標準k,i,j原地更新仍可用於負環偵測：所有有限值都有真walk見證，且不大於相應buffered候選，故不漏掉負對角；只是不同更新細節可得到不同暫存負數。

另算reachability R。只有滿足「存在k，D[k,k]<0、R[i,k]且R[k,j]」的(i,j)標為−∞。充分性是i先到負圈、重複任意多次再到j。必要性是：若i→j相關區域沒有負cycle，刪除其所有非負圈後，只剩有限多simple paths，最小成本有下界，不可能為−∞。不可達仍為∞；其餘pair的Floyd值維持有效。

標記後才交付結果，不把−∞再投入一般∞加法。 Johnson可直接拒絕整張有負環的圖；若要交付部分有限結果，則需這種pair層級語義。

不是整張圖都受負環影響

圖有1→2權2、2→3權−3、3→2權1、3→4權4，點5孤立。負圈2→3→2成本−2。

起點＼終點	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

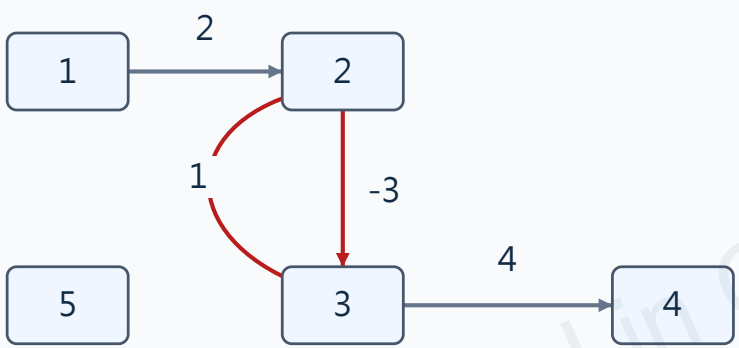
1可以進入負圈再離開到4，所以$\delta(1,4)=-\infty$；但沒有回1的路，$\delta(1,1)=0$。4、5不能進負圈，自己的零長距離不變。這比「發現負對角就把所有格標錯誤」更精確。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

不是整張圖都受負環影響 （續）

負環影響的是9個點對，並非整張矩陣

紅邊負圈2→3→2成本−2；紅格只標能進入且能離開負圈的點對。



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

1能進圈但不能回1，故δ(1,1)=0；4與5不能進圈。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transitive Closure：相同分解，不同代數

$R^{(0)}[i,j]=(i=j)$或存在$i\to j$；保留零長路徑，因此對角線true。把 min/+ 換成 OR/AND，得到

$R^{(k)}[i,j]=R^{(k-1)}[i,j]$ OR ($R^{(k-1)}[i,k]$ AND $R^{(k-1)}[k,j]$)。

有路徑時可刪除重複頂點，不管圈的權重；依是否使用中介k切開，兩方向涵蓋全部可達路徑。這是布林版本的正確性，不依賴無負cycle。原地更新也安全，因R[k,k]=true，而R[i,k] OR (R[i,k] AND true)仍是R[i,k]，row k同理。

六邊共同例的終值rows為 [1,1,1,1]、[0,1,1,1]、[0,1,1,1]、[0,1,1,1]。時間$\Theta(n^3)$布林操作，空間$\Theta(n^2)$布林值；bit packing可減儲存與利用word並行，但須明說word大小及採用的實作，不能無條件把bit操作當任意長字串一次完成。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

25.3 Johnson’ s algorithm for sparse graphs

Johnson要把每條負邊改成非負，同時保持同一對端點所有路徑的相對成本。給任意有限potential $h:V\to R$，定義 $\hat{w}(u,v)=w(u,v)+h(u)-h(v)$。對路徑$v_0,\cdots,v_r$相加，內部h消去：

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$。

對固定起終點，增加量完全相同，所以成本的嚴格大小與等值都保留，最短路徑**集合**相同；不同tie-breaking仍可能挑出不同predecessor tree。對cycle起終同點，增加量0，故負cycle也完全保留；重加權不可能把原負cycle「修成可解」。

反例：對每條邊一律加常數C，長r的路徑會多rC，不是固定端點常數，可能改變最短路徑。1→4直達權6、1→2權2、2→4權2，原本後者權4較短；每邊加3後直達9而兩段10，答案反轉。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Potential 的存在、計算與負環拒絕

新增super-source s，向每個原點加權0邊，且沒有邊回s。若原圖無負cycle，新圖也沒有；每個v可達，Bellman–Ford得到有限h(v)=δ(s,v)。對任一u→v，接上邊可得h(v)≤h(u)+w(u,v)，重排就是ŵ(u,v)≥0，這才允許Dijkstra。

可不用實際存s的n條邊：先設每個h[v]=0（等價先放鬆所有s→v），再同步掃原圖邊n−1輪，最後檢查還有無h[v]>h[u]+w(u,v)。它相當於從任一原點零成本起跑、最多n−1條原邊；無負cycle的最短路徑可取simple，因此足夠。有任意負cycle時，各邊potential不等式不可能同時成立，沿cycle相加會推出0≤負數，所以最後必拒絕。

共同例初始h=(0,0,0,0)；第一輪只有2→3把h[3]降為−2，得到(0,0,−2,0)；第二、三輪不變。六條邊依序重加權為4、9、0、6、1、1，全部非負。這裡的h不是從原頂點1起跑的距離；h[2]=0而δ(1,2)=4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Potential 的存在、計算與負環拒絕 （續）

Johnson 的 Potential 與六條重加權

h[1]=0		h[2]=0	h[3]=-2	h[4]=0
原邊	w	h[u]	h[v]	$\hat{w}=w+h[u]-h[v]$
1→2	4	0	0	4+(0)−(0)=4
1→3	7	0	-2	7+(0)−(-2)=9
2→3	-2	0	-2	-2+(0)−(-2)=0
2→4	6	0	0	6+(0)−(0)=6
3→4	3	-2	0	3+(-2)−(0)=1
4→2	1	0	0	1+(0)−(0)=1

綠格全非負；h來自新增零邊source，不是從頂點1出發的原距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個來源跑 Dijkstra，再還原距離

```
Johnson(G):
  h=前頁全零初始化的Bellman-Ford potential
  若仍可relax則回報負cycle並停止
  對每條原邊設 ŵ(u,v)=w(u,v)+h[u]-h[v]
  for source u in V:
    以非負ŵ在原圖頂點上做Dijkstra，得distHat[v]
    for v in V:
      D[u,v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

從1出發，初始distHat=(0,∞,∞,∞)。取出1後為(0,4,9,∞)；取出2後，3由9降4、4由∞降10；取出3後4由10降5；最後取4不改善2。故依序取出1:0、2:4、3:4、4:5（等距採編號小者先），終值(0,4,4,5)，還原為(0,4,2,5)。

其餘來源的distHat rows依序為 [∞,0,0,1]、[∞,2,0,1]、[∞,1,1,0]。各格減h[source]再加h[target]，得到前述Floyd最終矩陣。例如3→2原距離2−(−2)+0=4，4→3原距離1−0+(−2)=−1。不可達仍∞，不要對sentinel做有限數的還原。

每個來源跑 Dijkstra，再還原距離 （續）

非負圖的 Dijkstra 與原距離還原

重加權皆非負；格內是distHat。相同距離以編號小者先取。

動作	到1	到2	到3	到4
初始化	0	∞	∞	∞
取出1	0	4	9	∞
取出2	0	4	4	10
取出3	0	4	4	5
取出4	0	4	4	5

綠格 = 本輪嚴格改善；原距離 = distHat − h[1] + h[v]。

終值 (0,4,4,5) 還原為 (0,4,2,5)，與Floyd row1一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏性、Priority Queue 與輸出成本

全零potential實作初始化$O(n)$、$n-1$輪各掃m邊、最後再掃m邊：$O(nm+n)$。若直接在擴增圖做一般Bellman–Ford，則為$O((n+1)(m+n))$，不要忘記新加的n條邊。

Fibonacci heap的Dijkstra為$O(m+n \log(n+1))$攤銷時間；做n次並填n^2輸出，Johnson總$O(nm+n^2 \log(n+1))$。Binary heap則是$O(n(m+n)\log(n+1))$，不能引用前一個去掉m的log因子的界。額外圖與單趟狀態$O(n+m)$，若保存完整輸出另需$\Theta(n^2)$；逐row串流可省輸出常駐空間，但沒有交付少於n^2格的同一輸出。

$n=1000, m=5000$時：Floyd有10^9個(i,j,k)更新；potential至多約5×10^6次原邊掃描，binary-heap重複Dijkstra的量級項約$1000 \cdot 6000 \cdot \log_2 1000 \approx 5.98\times10^7$，Fibonacci-heap界的量級項則$5\times10^6+10^6 \cdot \log_2 1000 \approx 1.50\times10^7$。這些是不同基本操作的成本模型，**不是實測執行時間或可直接相除的加速比**。稠密$m=\Theta(n^2)$時Floyd簡單且$\Theta(n^3)$，Johnson的稀疏優勢不再成立。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 25。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study